Team Number: 26
Members: Meggie Chen (mc2894), Allison Newman (akn47), Andrea Wang (aw868)

I: Robot Design and Strategy Overview

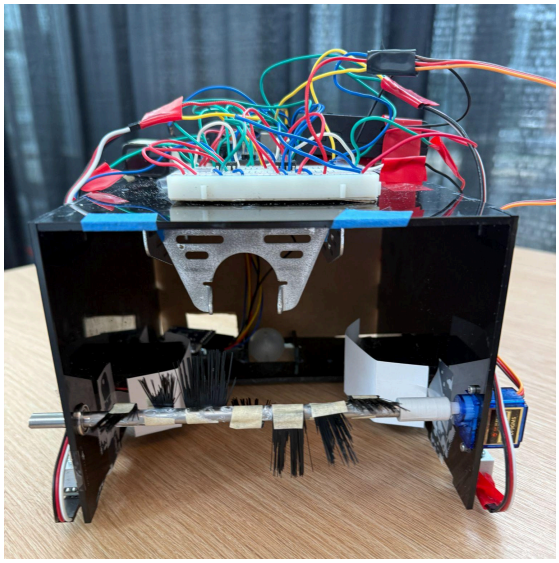


Fig. 1: Front View of Robot

Mechanical

Our robot consists of a 7.5 x 6.5 x 5 inch laser-cut acrylic frame, two continuous-servo-mounted wheels, and a free rolling ball at the back. There is a rotating shaft with brush attachments at the front used to scoop and retain blocks. Our robot does not use the metal boe-bot chassis. Our strategy was to use the larger acrylic frame to maximize our ability to surround and engulf blocks. With a larger surface area than the metal chassis, this frame was able to enclose blocks without deploying additional attachments.

Electrical

Our robot uses three continuous rotation servos: two control the wheels and one microservo controls the shaft. All three servos are connected to the 6V rail on our breadboard to maximize power output. With more torque to our microservo, we're able to use the brush to keep the blocks trapped within our robot once it's collected.

Software

Our robot utilizes two QTIs and a Roomba algorithm to move across the board. The QTIs are mounted on the left and right side of the robot. When both QTIs detect a black border, it moves backwards and turns right. When only the left QTI detects black, it turns slightly right. When only the right QTI detects black, it turns slightly left. If no QTI detects black, it keeps moving forward. At first, we planned to implement a color sensor that would detect when the robot

reached the center line and then cause the robot to turn right and scoop up blocks in the middle. However, we realized that that algorithm was finicky with the lighting and discoloration of the boards. So, we stuck to our Roomba algorithm that utilizes just QTI sensors.

II: Design Process Reflection

Milestones 1-3 were completed without the full mechanical assembly of our robot (e.g. custom frame, rotating brush) as these focused more on the code. During mechanical integration for Milestone 4, a major roadblock we encountered was that our brush mechanism did not work as intended; the provided servo could not provide enough torque to brush the blocks inside the robot. We changed the servo power from the Arduino (5V) to the 6V power rail and observed more power output, but it was still insufficient. Unfortunately, the cutout in the acrylic side panel was designed for the microservo, so we couldn't change the servo sizing. We decided to still keep the mechanism, as it was somewhat effective at keeping the blocks inside the frame of the robot after the robot drove over the blocks. Additionally, we observed issues with the color sensing portion of our code: we needed to retune the color frequencies on different days and the color sensor would return inaccurate readings over scratches in the board. We decided to scrap the color sensor since it was only used to drive up to the blue-yellow border. Instead, we hard-coded this distance.

III: Competition Analysis

Our robot made it to the semifinals, and placed 4th overall! This was a much better result than we expected. Our main strength was the custom chassis, which significantly improved our cube collection area (essentially the entire 8x8 square.) Additionally, our code was extremely simple—we only used 2 QTI sensors—but it did what it needed to by keeping us on the board. Many teams we competed against fell off the board or were much less effective at collecting, because their cubes would have to wrap around the chassis or stay bunched near the front (where they are prone to being lost), whereas we simply needed to drive over the cubes. The most significant issue we ran into was when we were pushed off the board by the other robot and could no longer collect more cubes or worse, cubes we had already collected were knocked out. It was easy for other robots to push us off because of our lightweight design that didn't stand up to some of the bulkier robots we went up against.

IV: Conclusions

If we could do this project over again, we would order our parts earlier so that we could see how the mechanical design actually works together. If we implemented our design sooner, we would've been able to test our rotating brush mechanism sooner and make adjustments. Design-wise, we could've made our robot heavier (e.g. using a heavier material for the frame)

so it could stand its ground better when facing another robot. We also could have spent more time thinking about how to keep the blocks inside our frame more permanently.

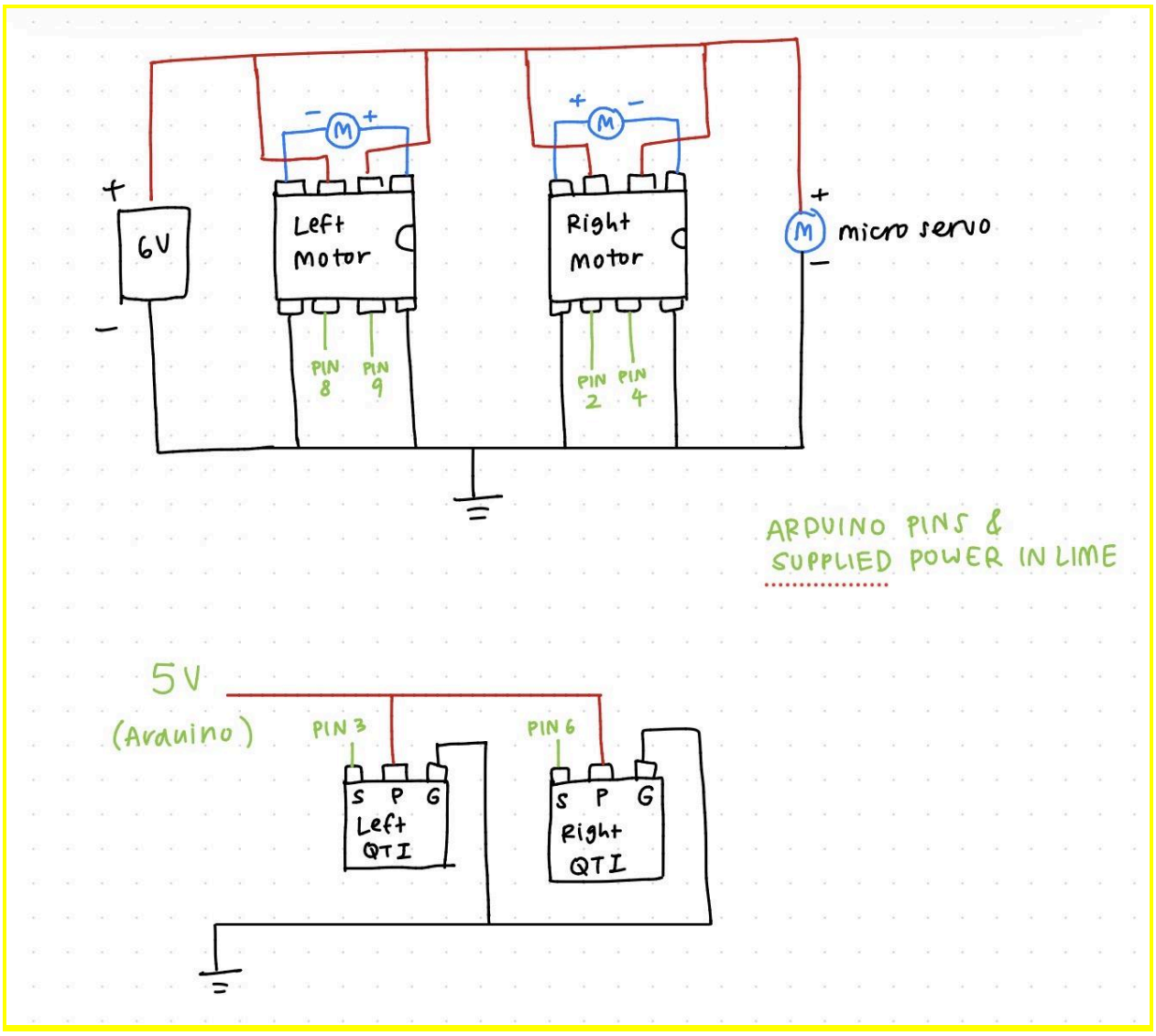
Our advice for students doing this project next year is to keep the design as simple as possible. Exploring creative ideas can be valuable, but it's important not to get too attached to a component and to be willing to replace features that aren't working with simpler, more reliable solutions. We were worried our robot was too simple but this ended up being an advantage since there were less variables for us to test; a lot of the more complex robots had interesting strategies that didn't work as intended. For example, one group deployed a barricade to block the opposing robot from crossing the board; however, because both the barricade and robot were lightweight, opponents could often push the barricade aside or into the robot itself—forcing them off the board. Another group implemented push buttons at the front of their robot which would drive their robot backward when activated (i.e. when they collided with another robot.) This was a cool idea in theory, but they ended up driving too far back and off the board.

Additionally, we would suggest leaning into custom-built parts to better tune the robot toward the competition objective. One of our main discussion topics during the design process was that the provided metal chassis was too low to the ground for effective cube collection, and that it seemed to be better suited toward previous years' objectives (e.g. moving cubes rather than collecting them.) This led us to design a custom frame that removed these issues.

Appendix A: Bill of Materials (BOM)

Component	Source	Quantity	Cost per Unit	Total Cost
Easy-Cut Strip Brush (7900T807)	McMaster	3 ft	$3.71/ft	$11.13
Ball Bearing (57155K376)	McMaster	1	$5.28/unit	$5.28
Laser Cut Acrylic Housing	RPL	2 6.5x7.5” panels, 2 5x7.5” panels	$0.05/inch cut + $0.50/part + $5/sheet	$17.30
3D Printed Shaft Collar	RPL	1	$1.00/part + $0.40/g	$1.60
3D Printed QTI Mounts	RPL	2	$1.00/part + $0.40/g	$3.50
Metal Shaft	Misc.	1	“Dumpster Dive”	$1.00
Cardboard	Misc.	1 5x7.5” panel	“Dumpster Dive”	$0.10
Rubber Bands	Misc.	2	Given by TA’s	$0.00
Total				$39.91

Appendix B: Circuit Diagram



Appendix C: CAD

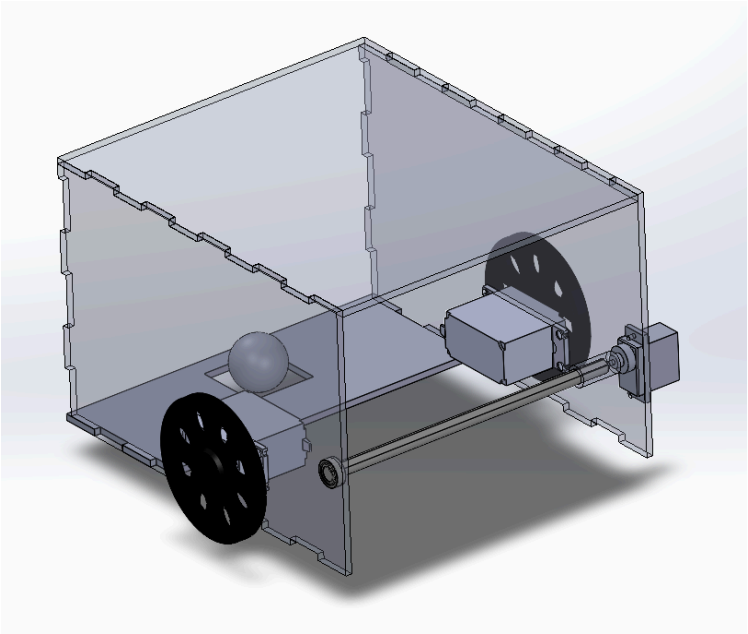


Fig. 2 Full CAD Assembly of Robot

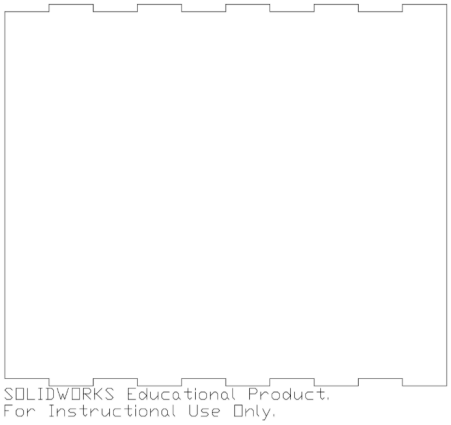


Fig. 3 Top Panel (Acrylic ⅛ inch Lasercut)

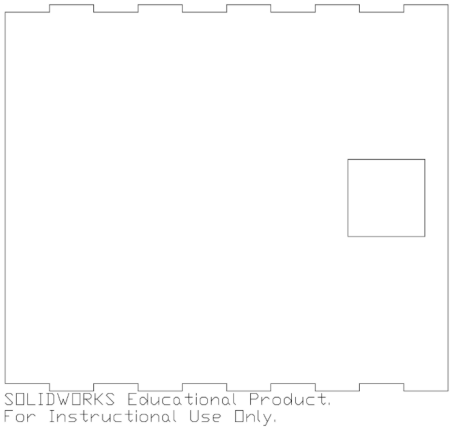


Fig. 4 Bottom Panel which we later dremelled to half the width (Acrylic ⅛ inch Lasercut)

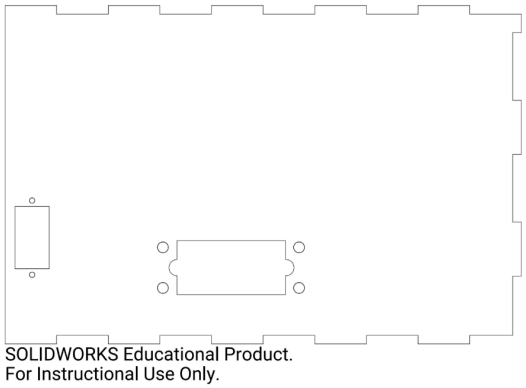


Fig. 5 Left Side Panel (Acrylic ⅛ inch Lasercut)

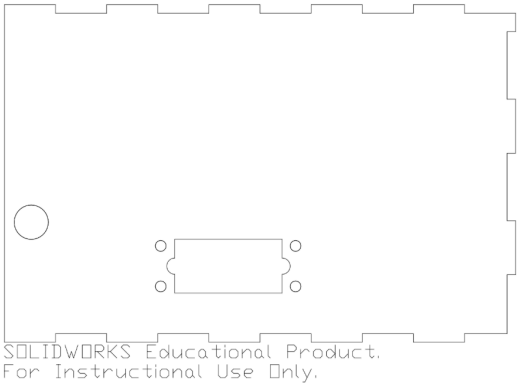


Fig. 6 Right Side Panel (Acrylic ⅛ inch Lasercut)

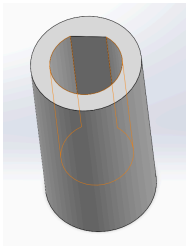


Fig. 7 Servo Shaft Connector

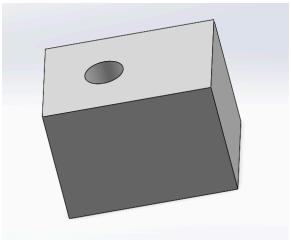


Fig. 8 QTI Mount

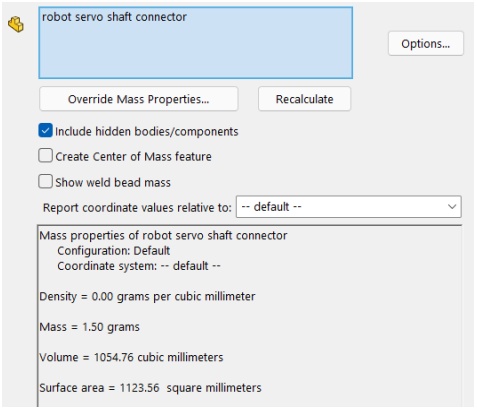


Fig. 9 Solidworks Mass Properties for Servo Shaft Connector (1.5g)

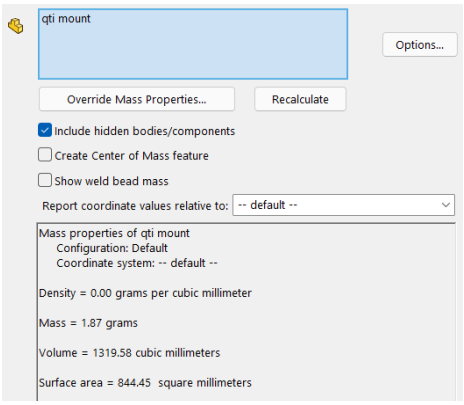
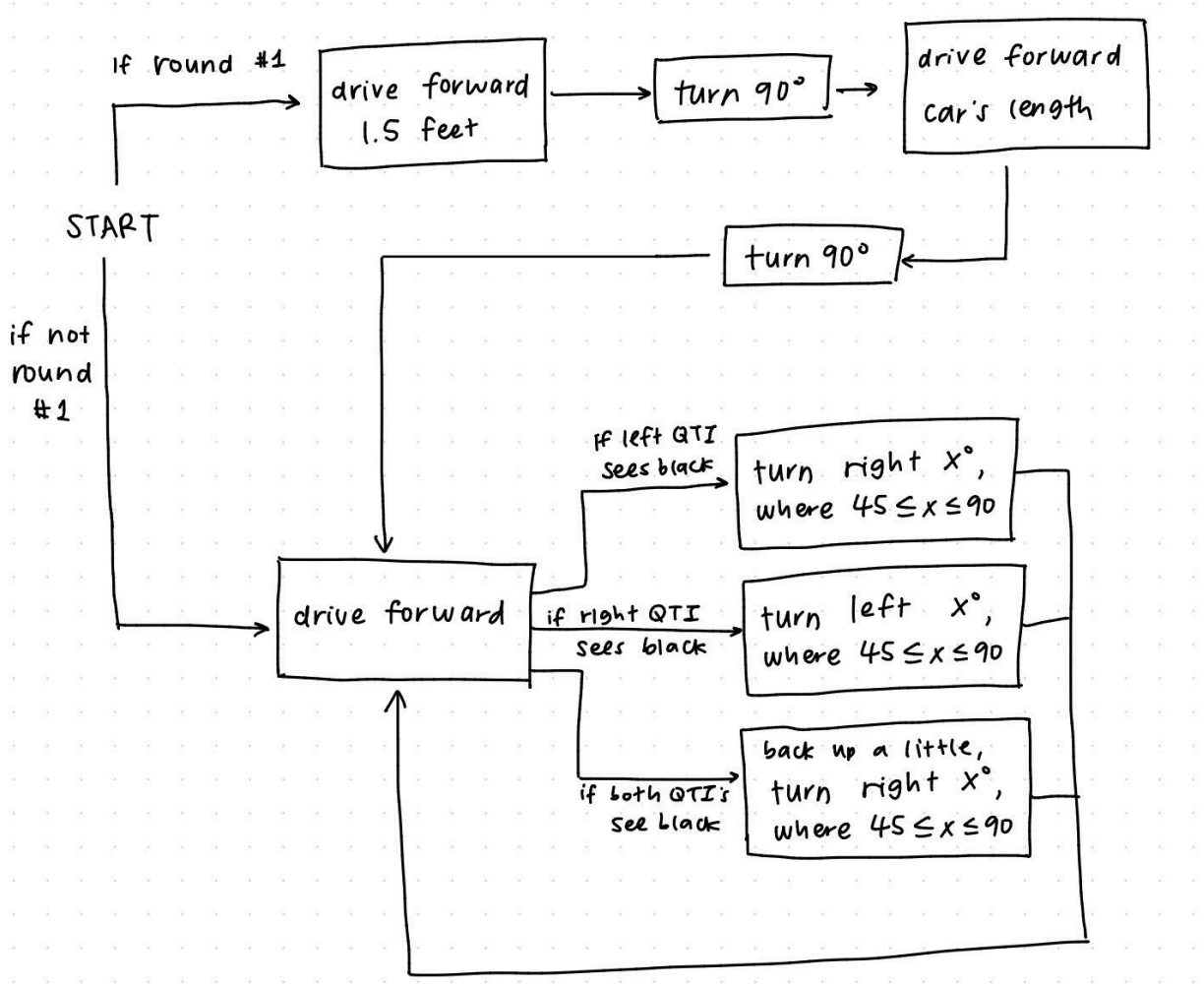


Fig. 10 Mass Properties for QTI Mount (1.87g)

Appendix D:

NOTE: This is our original flowchart, but the “if round 1” logic no longer follows since the motion was hard coded in rather than dependent on QTI’s. This is reflected in our code.



Appendix E:

References: Gemini AI for debugging purposes, continuous rotation servo code

```
int QTIState;
// initialize variable ‘QTIState’ where integer values correspond to a
different state as described below:
// 0 - neither QTI is black, 1 - both QTIs black, 2 - left QTI is black, 3 -
right QTI is black

int main(void){ // setup code that only runs once
  // set pins 8 and 9 as outputs for "left" motor and 11 for servo
  DDRB = 0b00000011;
  //set pins 2 and 4 as outputs for the right motor
```

```
  DDRD = 0b00010100;
  // Set Pin 11 (PB3) as output for cont rotation servo
  DDRB |= 0b00001000;

  // Timer 2 config for servo
  // Fast PWM, TOP at 0xFF, Prescaler 1024 (~61Hz)
   TCCR2A = (1 << COM2A1) | (1 << WGM21) | (1 << WGM20);
   TCCR2B = (1 << CS22) | (1 << CS21) | (1 << CS20);

   OCR2A = 15; // setting servo spin speed and direction

   sei(); // enable interrupts
   Serial.begin(9600); // enable serial for debugging

   int round = 0; // initialize counter variable 'round' to keep track of which
round the robot is on; after a certain number of rounds (or interruption) it
will roomba around the board


   while(1){

      QTIState = getQTIState(); //retrieve the QTI states

      if(round < 1){
        drive_forward_delay(6500); //drive forward half length of the board
        drive_right_delay(975); //turn 90 degrees
        drive_forward_delay(1500); //drive forward a car length
        drive_right_delay(975); //turn 90 degrees

         }
        // will transition to roomba after this
         round = round + 1;
      }
      // QTI setup
      // Charge capacitor for 1ms
      //Serial.println(QTIState);

      else{
         roomba();
      }
   }
}

   void driveForwardRound(){
      while(QTIState != 1){
         if(QTIState == 2){
            _delay_ms(200);
            QTIState = getQTIState();
```

```
            if(QTIState == 2){
               roomba();
            }
         }
         else if (QTIState == 3){
            _delay_ms(200);
            QTIState = getQTIState();

            if(QTIState == 3){
               roomba();
            }
         }

         QTIState = getQTIState();
         drive_forward();
      }
   }

   // 'roomba' function that avoids the border by turning away at a random
angle
   void roomba(){
      while(1){
      // randomize the turn angle
         int angle = random(400, 700);
      // continuously get the QTI state
         QTIState = getQTIState();
      // if both QTIs detect black, the robot will back up and turn away
         if(QTIState == 1){
            drive_backward_delay(300);
            drive_right_delay(angle);
         }
      // if neither QTI detects black, the robot will continue driving forward
         else if(QTIState == 0){
            drive_forward_delay(300);
         }
      // if the left QTI detects black, the robot will turn right
         else if(QTIState == 2){
            drive_right_delay(angle);
         }
      // if the right QTI detects black, the robot will turn left
         else if(QTIState == 3){
            drive_left_delay(angle);
         }
      }
   }

// function 'getQTIState' returns an integer code representing the QTI state
int getQTIState(){
   DDRD |= 0b01001000; // pins 3 (left) and 5 (right) used for QTI sensors
```

```
   PORTD |= 0b01001000;
   _delay_ms(1);
   //Discharge capacitor for 1ms
   DDRD &= 0b10110111;
   _delay_ms(1);
   if(!(PIND &= 0b01000000)&&!(PIND &= 0b00001000)){ //if neither sensor is
black
      return(0);
   }
   else if((PIND &= 0b01000000) && (PIND &= 0b00001000)){ //if both sensors
black
      return(1);
   }
   else if(PIND &= 0b01000000){ //if left sensor black
      return(2);
   }
   else if(PIND &= 0b00001000){ //if right sensor black
      return(3);
   }else{
     return(4);
   }
}

// function to drive forward for a specified amount of time (in milliseconds)
void drive_forward_delay(int t){
   // code to control appropriate pins here (both motors forward)
   PORTB &= 0b11111110;
   PORTB |= 0b00000010;
   PORTD &= 0b11111011;
   PORTD |= 0b00010000;
   while (t > 0) {
   _delay_ms(1); // 1 is a constant, so the compiler is happy
   t--;
   }
}

// function to drive forward, without specifying time
void drive_forward(){
   PORTB &= 0b11111110;
   PORTB |= 0b00000010;
   PORTD &= 0b11111011;
   PORTD |= 0b00010000;
}

// function to drive backward for a specified amount of time (in milliseconds)
void drive_backward_delay(int t){
// code to control appropriate pins here (both motors backward)
   PORTB &= 0b11111101;
   PORTB |= 0b00000001;
```

```
   PORTD &= 0b11101111;
   PORTD |= 0b00000100;
   while (t > 0) {
      _delay_ms(1); // 1 is a constant, so the compiler is happy
      t--;
   }
}

// function to turn right for a specified amount of time (in milliseconds)
void drive_right_delay(int t){
// code to control appropriate pins here (right motor forward, left motor
backward)
   PORTB &= 0b11111110;
   PORTB |= 0b00000010;
   PORTD &= 0b11101111;
   PORTD |= 0b00000100;
   while (t > 0) {
      _delay_ms(1); // 1 is a constant, so the compiler is happy
      t--;
   }
}

// function to turn left for a specified amount of time (in milliseconds)
void drive_left_delay(int t){
// code to control appropriate pins here (left motor forward, right motor
backward)
   PORTB &= 0b11111101;
   PORTB |= 0b00000001;
   PORTD &= 0b11111011;
   PORTD |= 0b00010000;
   while (t > 0) {
      _delay_ms(1); // 1 is a constant, so the compiler is happy
      t--;
   }
}

// function to stop motors
void stop_motors() {
// turn off all pins
   PORTB &= ~0b00000011;
   PORTD &= ~0b00010100;
   // Small delay so the robot actually stays still for a moment
   int wait = 500; // define an arbitrarily large integer variable 'wait' to
loop through
   while(wait > 0) {
   _delay_ms(5000);
   wait--;
   }
}
```